

Chapter 4: From Gradient Boosting to XGBoost

XGBoost is a unique form of gradient boosting with several distinct advantages, which will be explained in [Chapter 5, XGBoost Unveiled](#). In order to understand the advantages of XGBoost over traditional gradient boosting, you must first learn how traditional gradient boosting works. The general structure and hyperparameters of traditional gradient boosting are incorporated by XGBoost. In this chapter, you will discover the power behind gradient boosting, which is at the core of XGBoost.

In this chapter, you will build gradient boosting models from scratch before comparing gradient boosting models and errors with previous results. In particular, you will focus on the **learning rate** hyperparameter to build powerful gradient boosting models that include XGBoost. Finally, you will preview a case study on exoplanets highlighting the need for faster algorithms, a critical need in the world of big data that is satisfied by XGBoost.

In this chapter, we will be covering the following main topics:

- From bagging to boosting
- How gradient boosting works
- Modifying gradient boosting hyperparameters
- Approaching big data – gradient boosting versus XGBoost

Technical requirements

The code for this chapter is available at <https://github.com/PacktPublishing/Hands-On-Gradient-Boosting-with-XGBoost-and-Scikit-learn/tree/master/Chapter04>.

From bagging to boosting

In [Chapter 3, Bagging with Random Forests](#), you learned why ensemble machine learning algorithms such as random forests make better predictions by combining many machine learning models into one. Random forests are classified as bagging algorithms because they take the aggregates of bootstrapped samples (decision trees).

Boosting, by contrast, learns from the mistakes of individual trees. The general idea is to adjust new trees based on the errors of previous trees.

In boosting, correcting errors for each new tree is a distinct approach from bagging. In a bagging model, new trees pay no attention to previous trees. Also, new trees are built from scratch using bootstrapping, and the final model aggregates all individual trees. In boosting,

however, each new tree is built from the previous tree. The trees do not operate in isolation; instead, they are built on top of one another.

Introducing AdaBoost

AdaBoost is one of the earliest and most popular boosting models. In AdaBoost, each new tree adjusts its weights based on the errors of the previous trees. More attention is paid to predictions that went wrong by adjusting weights that affect those samples at a higher percentage. By learning from its mistakes, AdaBoost can transform weak learners into strong learners. A weak learner is a machine learning algorithm that barely performs better than chance. By contrast, a stronger learner has learned a considerable amount from data and performs quite well.

The general idea behind boosting algorithms is to transform weak learners into strong learners. A weak learner is hardly better than random guessing. But there is a purpose behind the weak start. Building on this general idea, boosting works by focusing on iterative error correction, *not* by establishing a strong baseline model. If the base model is too strong, the learning process is necessarily limited, thereby undermining the general strategy behind boosting models.

Weak learners are transformed into strong learners through hundreds of iterations. In this sense, a small edge goes a long way. In fact, boosting has been one of the best general machine learning strategies in terms of producing optimal results for the past couple of decades.

A detailed study of AdaBoost is beyond the scope of this book. Like many scikit-learn models, it's straightforward to implement AdaBoost in practice.

The **AdaBoostRegressor** and **AdaBoostClassifier** algorithms may be downloaded from the `sklearn.ensemble` library and fit to any training set. The most important AdaBoost hyperparameter is `n_estimators`, the number of trees (iterations) required to create a strong learner.

Note

For further information on AdaBoost, check out the official documentation at <https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.AdaBoostClassifier.html> for classifiers and <https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.AdaBoostRegressor.html> for regressors.

We will now move on to gradient boosting, a strong alternative to AdaBoost with a slight edge in performance.

Distinguishing gradient boosting

Gradient boosting uses a different approach than AdaBoost. While gradient boosting also adjusts based on incorrect predictions, it takes this idea one step further: gradient boosting fits each new tree entirely based on the errors of the previous tree's predictions. That is, for each new tree, gradient boosting looks at the mistakes and then builds a new tree completely around these mistakes. The new tree doesn't care about the predictions that are already correct.

Building a machine learning algorithm that solely focuses on the errors requires a comprehensive method that sums errors to make accurate final predictions. This method leverages residuals, the difference between the model's predictions and actual values. Here is the general idea:

Gradient boosting computes the residuals of each tree's predictions and sums all the residuals to score the model.

It's essential to understand **computing** and **summing residuals** as this idea is at the core of XGBoost, an advanced version of gradient boosting. When you build your own version of gradient boosting, the process of computing and summing residuals will become clear. In the next section, you will build your own version of a gradient boosting model. First, let's learn in detail how gradient boosting works.

How gradient boosting works

In this section, we will look under the hood of gradient boosting and build a gradient boosting model from scratch by training new trees on the errors of the previous trees. The key mathematical idea here is the residual. Next, we will obtain the same results using scikit-learn's gradient boosting algorithm.

Residuals

The residuals are the difference between the errors and the predictions of a given model. In statistics, residuals are commonly analyzed to determine how good a given linear regression model fits the data.

Consider the following examples:

1. Bike rentals
 - a) *Prediction*: 759
 - b) *Result*: 799
 - c) *Residual*: $799 - 759 = 40$

2. Income

a) *Prediction*: 100,000

b) *Result*: 88,000

c) *Residual*: $88,000 - 100,000 = -12,000$

As you can see, residuals tell you how far the model's predictions are from reality, and they may be positive or negative.

Here is a visual example displaying the residuals of a **linear regression** line:

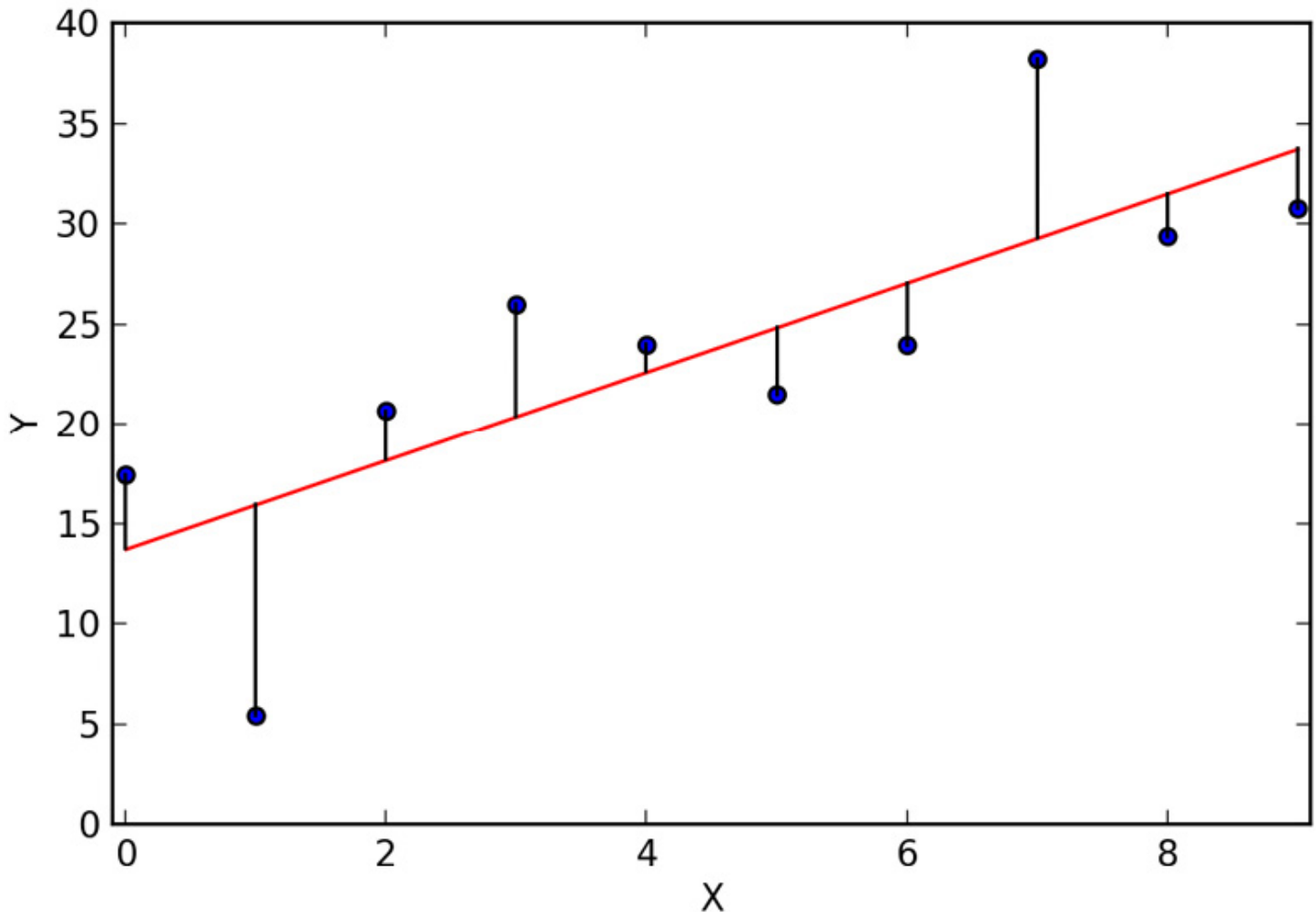


Figure 4.1 – Residuals of a linear regression line

The goal of linear regression is to minimize the square of the residuals. As the graph reveals, a visual of the residuals indicates how well the line fits the data. In statistics, linear regression analysis is often performed by graphing the residuals to gain deeper insight into the data.

In order to build a gradient boosting algorithm from scratch, we will compute the residuals of each tree and fit a new model to the residuals. Let's do this now.

Learning how to build gradient boosting models from scratch

Building a gradient boosting model from scratch will provide you with a deeper understanding of how gradient boosting works in code. Before building a model, we need to access data and prepare it for machine learning.

Processing the bike rentals dataset

We continue with the bike rentals dataset to compare new models with the previous models:

1. We will start by importing **pandas** and **numpy**. We will also add a line to silence any warnings:

```
import pandas as pd
import numpy as np
import warnings
warnings.filterwarnings('ignore')
```

2. Now, load the **bike_rentals_cleaned** dataset and view the first five rows:

```
df_bikes = pd.read_csv('bike_rentals_cleaned.csv')
df_bikes.head()
```

Your output should look like this:

	instant	season	yr	mnth	holiday	weekday	workingday	weathersit	temp	atemp	hum	windspeed	cnt
0	1	1.0	0.0	1.0	0.0	6.0	0.0	2	0.344167	0.363625	0.805833	0.160446	985
1	2	1.0	0.0	1.0	0.0	0.0	0.0	2	0.363478	0.353739	0.696087	0.248539	801
2	3	1.0	0.0	1.0	0.0	1.0	1.0	1	0.196364	0.189405	0.437273	0.248309	1349
3	4	1.0	0.0	1.0	0.0	2.0	1.0	1	0.200000	0.212122	0.590435	0.160296	1562
4	5	1.0	0.0	1.0	0.0	3.0	1.0	1	0.226957	0.229270	0.436957	0.186900	1600

Figure 4.2 – First five rows of Bike Rental Dataset

3. Now, split the data into **x** and **y**. Then, split **x** and **y** into training and test sets:

```
X_bikes = df_bikes.iloc[:, :-1]
y_bikes = df_bikes.iloc[:, -1]
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X_bikes, y_bikes,
                                                    random_state=2)
```

It's time to build a gradient boosting model from scratch!

Building a gradient boosting model from scratch

Here are the steps for building a gradient boosting machine learning model from scratch:

1. Fit the data to the decision tree: You may use a decision tree stump, which has a **max_depth** value of 1, or a decision tree with a **max_depth** value of 2 or 3. The initial decision tree, called a **base learner**, should not be fine-tuned for accuracy. We want a model that focuses on learning from errors, not a model that relies heavily on the base learner.

Initialize a decision tree with `max_depth=2` and fit it on the training set as `tree_1`, since it's the first tree in our ensemble:

```
from sklearn.tree import DecisionTreeRegressor
tree_1 = DecisionTreeRegressor(max_depth=2, random_state=2)
tree_1.fit(X_train, y_train)
```

2. Make predictions with the training set: Instead of making predictions with the test set, predictions in gradient boosting are initially made with the training set. Why? To compute the residuals, we need to compare the predictions while still in the training phase. The test phase of the model build comes at the end, after all the trees have been constructed. The predictions of the training set for the first round are obtained by adding the `predict` method to `tree_1` with `X_train` as the input:

```
y_train_pred = tree_1.predict(X_train)
```

3. Compute the residuals: The residuals are the differences between the predictions and the target column. The predictions of `X_train`, defined here as `y_train_pred`, are subtracted from `y_train`, the target column, to compute the residuals:

```
y2_train = y_train - y_train_pred
```

Note

The residuals are defined as `y2_train` because they are the new target column for the next tree.

4. Fit the new tree on the residuals: Fitting a new tree on the residuals is different than fitting a model on the training set. The primary difference is in the predictions. In the bike rentals dataset, when fitting a new tree on the residuals, we should progressively get smaller numbers.

Initialize a new tree and fit it on `X_train` and the residuals, `y2_train`:

```
tree_2 = DecisionTreeRegressor(max_depth=2, random_state=2)
tree_2.fit(X_train, y2_train)
```

5. Repeat steps 2-4: As the process continues, the residuals should gradually approach 0 from the positive and negative direction. The iterations continue for the number of estimators, `n_estimators`.

Let's repeat the process for a third tree as follows:

```
y2_train_pred = tree_2.predict(X_train)
y3_train = y2_train - y2_train_pred
tree_3 = DecisionTreeRegressor(max_depth=2, random_state=2)
tree_3.fit(X_train, y3_train)
```

This process may continue for dozens, hundreds, or thousands of trees. Under normal circumstances, you would certainly keep going. It will take more than a few trees to transform a weak learner into a strong learner. Since our goal is to understand how gradient boosting works behind the scenes, however, we will move on now that the general idea has been covered.

6. Sum the results: Summing the results requires making predictions for each tree with the test set as follows:

```
y1_pred = tree_1.predict(X_test)
y2_pred = tree_2.predict(X_test)
y3_pred = tree_3.predict(X_test)
```

Since the predictions are positive and negative differences, summing the predictions should result in predictions that are closer to the target column as follows:

```
y_pred = y1_pred + y2_pred + y3_pred
```

7. Lastly, let's compute the **mean squared error (MSE)** to obtain the results as follows:

```
from sklearn.metrics import mean_squared_error as MSE  
MSE(y_test, y_pred)**0.5
```

Here is the expected output:

```
911.0479538776444
```

Not bad for a weak learner that isn't yet strong! Now let's try to obtain the same result using scikit-learn.

Building a gradient boosting model in scikit-learn

Let's see whether we can obtain the same result as in the previous section using scikit-learn's **GradientBoostingRegressor**. This may be done through a few hyperparameter adjustments. The advantage of using **GradientBoostingRegressor** is that it's much faster to build and easier to implement:

1. First, import the regressor from the **sklearn.ensemble** library:

```
from sklearn.ensemble import GradientBoostingRegressor
```

2. When initializing **GradientBoostingRegressor**, there are several important hyperparameters. To obtain the same results, it's essential to match **max_depth=2** and **random_state=2**. Furthermore, since there are only three trees, we must have **n_estimators=3**. Finally, we must set the **learning_rate=1.0** hyperparameter. We will have much to say about **learning_rate** shortly:

```
gbr = GradientBoostingRegressor(max_depth=2, n_estimators=3,  
                                random_state=2, learning_rate=1.0)
```

3. Now that the model has been initialized, it can be fit on the training data and scored against the test data:

```
gbr.fit(X_train, y_train)  
y_pred = gbr.predict(X_test)  
MSE(y_test, y_pred)**0.5
```

The result is as follows:

```
911.0479538776439
```

The result is the same to 11 decimal places!

Recall that the point of gradient boosting is to build a model with enough trees to transform a weak learner into a strong learner. This is easily done by changing **n_estimators**, the number of iterations, to a much larger number.

4. Let's build and score a gradient boosting regressor with 30 estimators:

```
gbr = GradientBoostingRegressor(max_depth=2, n_estimators=30,  
                                random_state=2, learning_rate=1.0)  
gbr.fit(X_train, y_train)  
y_pred = gbr.predict(X_test)  
MSE(y_test, y_pred)**0.5
```


The result is as follows:

```
857.1072323426944
```

The score is an improvement. Now let's look at 300 estimators:

```
gbr = GradientBoostingRegressor(max_depth=2, n_estimators=300,  
    random_state=2, learning_rate=1.0)  
gbr.fit(X_train, y_train)  
y_pred = gbr.predict(X_test)  
MSE(y_test, y_pred)**0.5
```

The result is this:

```
936.3617413678853
```

This is a surprise! The score has gotten worse! Have we been misled? Is gradient boosting not all that it's cracked up to be?

Whenever you get a surprise result, it's worth double-checking the code. Now, we changed `learning_rate` without saying much about it. So, what happens if we remove `learning_rate=1.0` and use the scikit-learn defaults?

Let's find out:

```
gbr = GradientBoostingRegressor(max_depth=2, n_estimators=300, random_state=2)  
gbr.fit(X_train, y_train)  
y_pred = gbr.predict(X_test)  
MSE(y_test, y_pred)**0.5
```

The result is this:

```
653.7456840231495
```

Incredible! By using the scikit-learn default for the `learning_rate` hyperparameter, the score has changed from **936** to **654**.

In the next section, we will learn more about the different gradient boosting hyperparameters with a focus on the `learning_rate` hyperparameter.

Modifying gradient boosting hyperparameters

In this section, we will focus on the `learning_rate`, the most important gradient boosting hyperparameter, with the possible exception of `n_estimators`, the number of iterations or trees in the model. We will also survey some tree hyperparameters, and `subsample`, which results in **stochastic gradient boosting**. In addition, we will use `RandomizedSearchCV` and compare results with XGBoost.

`learning_rate`

In the last section, changing the `learning_rate` value of `GradientBoostingRegressor` from `1.0` to scikit-learn's default, which is `0.1`, resulted in enormous gains.

`learning_rate`, also known as the *shrinkage*, shrinks the contribution of individual trees so that no tree has too much influence when building the model. If an entire ensemble is built from the errors of one base learner, without careful adjustment of hyperparameters, early trees in the model can have too much influence on subsequent development. `learning_rate` limits the influence of individual trees. Generally speaking, as `n_estimators`, the number of trees, goes up, `learning_rate` should go down.

Determining an optimal `learning_rate` value requires varying `n_estimators`. First, let's hold `n_estimators` constant and see what `learning_rate` does on its own. `learning_rate` ranges from 0 to 1. A `learning_rate` value of 1 means that no adjustments are made. The default value of `0.1` means that the tree's influence is weighted at 10%.

Here is a reasonable range to start with:

```
learning_rate_values = [0.001, 0.01, 0.05, 0.1, 0.15, 0.2, 0.3, 0.5, 1.0]
```

Next, we will loop through the values by building and scoring a new `GradientBoostingRegressor` to see how the scores compare:

```
for value in learning_rate_values:
    gbr = GradientBoostingRegressor(max_depth=2, n_estimators=300,
    random_state=2, learning_rate=value)
    gbr.fit(X_train, y_train)
    y_pred = gbr.predict(X_test)
    rmse = MSE(y_test, y_pred)**0.5
    print('Learning Rate:', value, ', Score:', rmse)
```

The learning rate values and scores are as follows:

```
Learning Rate: 0.001 , Score: 1633.0261400367258
Learning Rate: 0.01 , Score: 831.5430182728547
Learning Rate: 0.05 , Score: 685.0192988749717
Learning Rate: 0.1 , Score: 653.7456840231495
Learning Rate: 0.15 , Score: 687.666134269379
Learning Rate: 0.2 , Score: 664.312804425697
Learning Rate: 0.3 , Score: 689.4190385930236
Learning Rate: 0.5 , Score: 693.8856905068778
Learning Rate: 1.0 , Score: 936.3617413678853
```

As you can see from the output, the default `learning_rate` value of `0.1` gives the best score for 300 trees.

Now let's vary `n_estimators`. Using the preceding code, we can generate `learning_rate` plots with `n_estimators` of 30, 300, and 3,000 trees, as shown in the following figure:

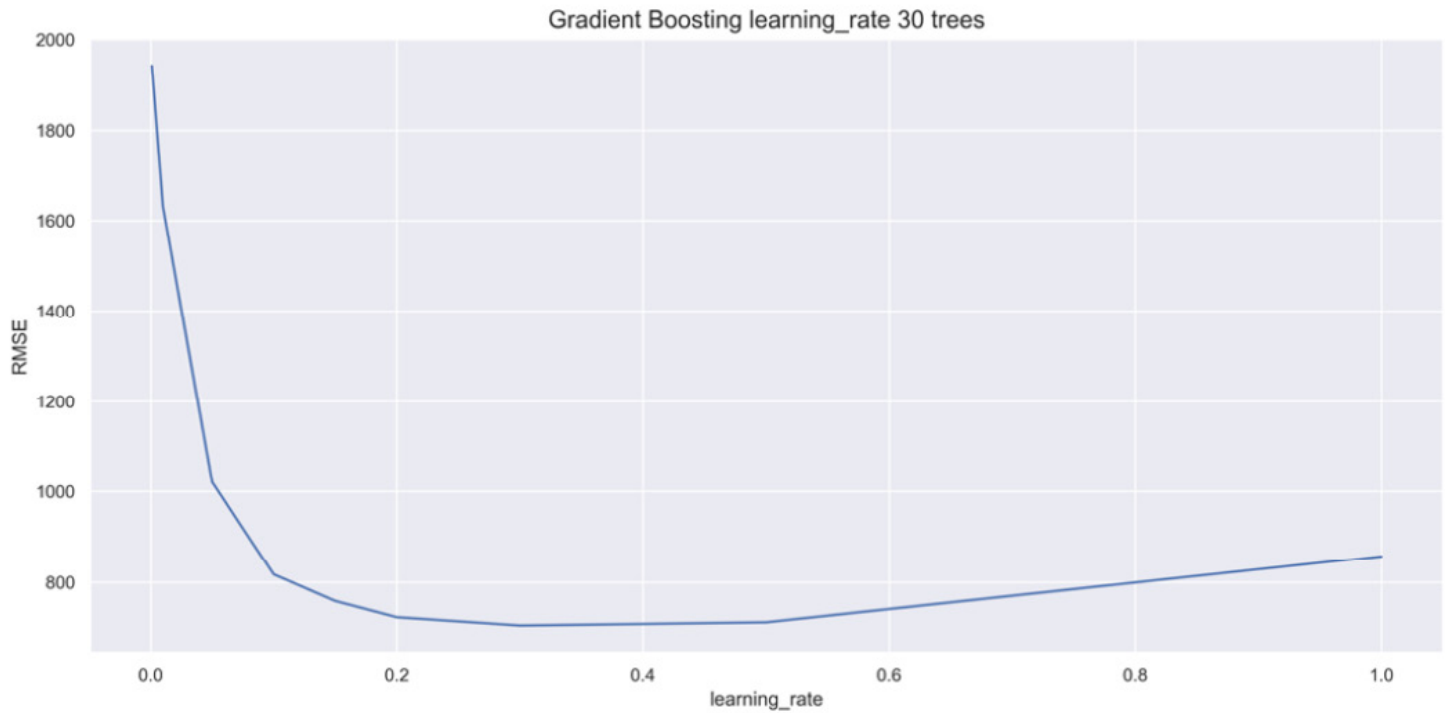


Figure 4.3 – `learning_rate` plot for 30 trees

As you can see, with 30 trees, the `learning_rate` value peaks at around 0.3.

Now, let's take a look at the `learning_rate` plot for 3,000 trees:

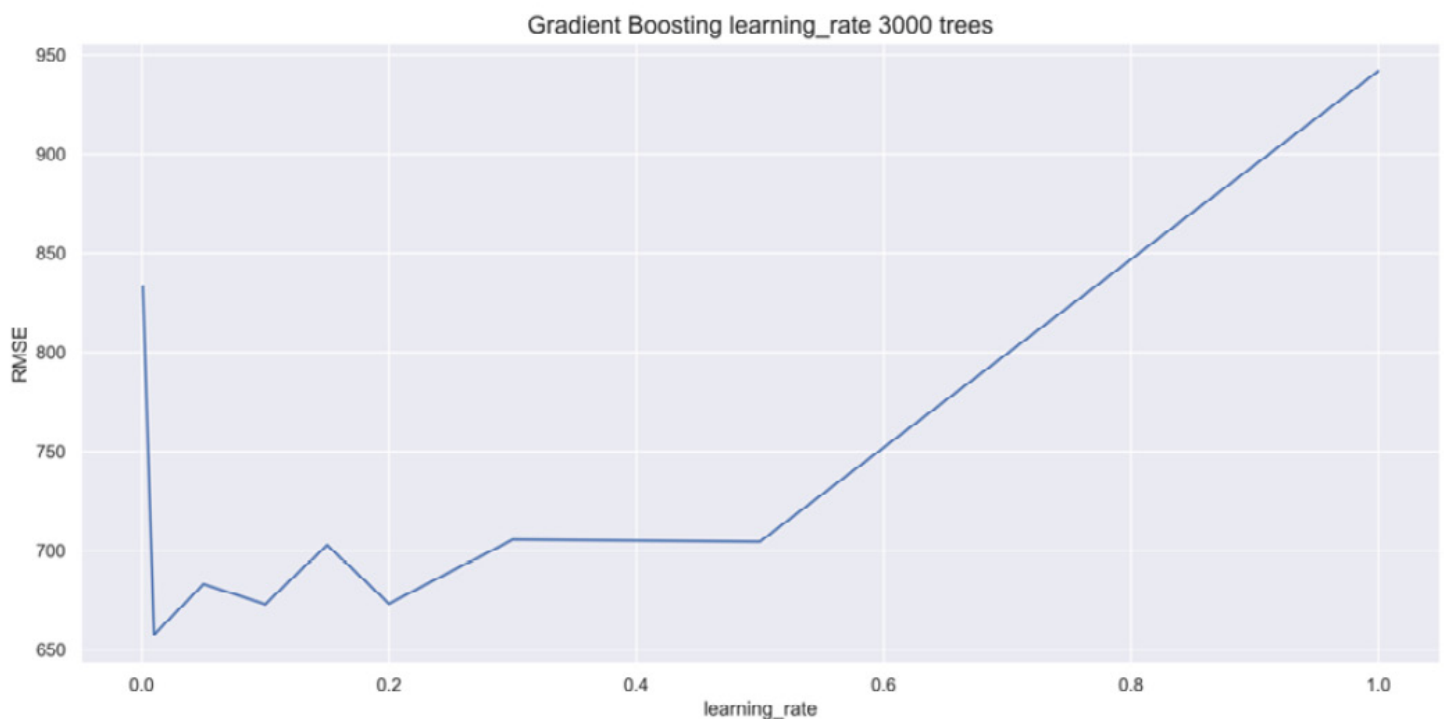


Fig 4.4 -- learning_rate plot for 3,000 trees

With 3,000 trees, the `learning_rate` value peaks at the second value, which is given as 0.05.

These graphs highlight the importance of tuning `learning_rate` and `n_estimators` together.

Base learner

The initial decision tree in the gradient boosting regressor is called the **base learner** because it's at the base of the ensemble. It's the first learner in the process. The term *learner* here is indicative of a *weak learner* transforming into a *strong learner*.

Although base learners need not be fine-tuned for accuracy, as covered in [Chapter 2, Decision Trees in Depth](#), it's certainly possible to tune base learners for gains in accuracy.

For instance, we can select a `max_depth` value of 1, 2, 3, or 4 and compare results as follows:

```
depths = [None, 1, 2, 3, 4]
for depth in depths:
    gbr = GradientBoostingRegressor(max_depth=depth, n_estimators=300,
    random_state=2)
    gbr.fit(X_train, y_train)
    y_pred = gbr.predict(X_test)
    rmse = MSE(y_test, y_pred)**0.5
    print('Max Depth:', depth, ', Score:', rmse)
```

The result is as follows:

```
Max Depth: None , Score: 867.9366621617327
Max Depth: 1 , Score: 707.8261886858736
Max Depth: 2 , Score: 653.7456840231495
Max Depth: 3 , Score: 646.4045923317708
Max Depth: 4 , Score: 663.048387855927
```

A `max_depth` value of 3 gives the best results.

Other base learner hyperparameters, as covered in [Chapter 2, Decision Trees in Depth](#), may be tuned in a similar manner.

subsample

subsample is a subset of samples. Since samples are the rows, a subset of rows means that all rows may not be included when building each tree. By changing `subsample` from 1.0 to a smaller decimal, trees only select that percentage of samples during the build phase. For example, `subsample=0.8` would select 80% of samples for each tree.

Continuing with `max_depth=3`, we try a range of subsample percentages to improve results:

```
samples = [1, 0.9, 0.8, 0.7, 0.6, 0.5]
for sample in samples:
    gbr = GradientBoostingRegressor(max_depth=3, n_estimators=300,
    subsample=sample, random_state=2)
    gbr.fit(X_train, y_train)
    y_pred = gbr.predict(X_test)
    rmse = MSE(y_test, y_pred)**0.5
    print('Subsample:', sample, ', Score:', rmse)
```

The result is as follows:

```
Subsample: 1 , Score: 646.4045923317708
Subsample: 0.9 , Score: 620.1819001443569
Subsample: 0.8 , Score: 617.2355650565677
Subsample: 0.7 , Score: 612.9879156983139
Subsample: 0.6 , Score: 622.6385116402317
Subsample: 0.5 , Score: 626.9974073227554
```

A `subsample` value of `0.7` with 300 trees and `max_depth` of 3 produces the best score yet.

When `subsample` is not equal to `1.0`, the model is classified as **stochastic gradient descent**, where *stochastic* indicates that some randomness is inherent in the model.

RandomizedSearchCV

We have a good working model, but we have not yet performed a grid search, as covered in [Chapter 2, Decision Trees in Depth](#). Our preliminary analysis indicates that a grid search centered around `max_depth=3`, `subsample=0.7`, `n_estimators=300`, and `learning_rate = 0.1` is a good place to start. We have already shown that as `n_estimators` goes up, `learning_rate` should go down:

1. Here is a possible starting point:

```
params={'subsample':[0.65, 0.7, 0.75],
        'n_estimators':[300, 500, 1000],
        'learning_rate':[0.05, 0.075, 0.1]}
```

Since `n_estimators` is going up from the starting value of 300, `learning_rate` is going down from the starting value of `0.1`. Let's keep `max_depth=3` to limit the variance. With 27 possible combinations of hyperparameters, we use `RandomizedSearchCV` to try 10 of these combinations in the hopes of finding a good model.

Note

While 27 combinations are feasible with `GridSearchCV`, at some point you will end up with too many possibilities and `RandomizedSearchCV` will become essential. We use `RandomizedSearchCV` here for practice and to speed up computations.

2. Let's import `RandomizedSearchCV` and initialize a gradient boosting model:

```
from sklearn.model_selection import RandomizedSearchCV
```

```
gbr = GradientBoostingRegressor(max_depth=3, random_state=2)
```

3. Next, initialize **RandomizedSearchCV** with **gbr** and **params** as inputs in addition to the number of iterations, the scoring, and the number of folds. Recall that **n_jobs=-1** may speed up computations and **random_state=2** ensures the consistency of results:

```
rand_reg = RandomizedSearchCV(gbr, params, n_iter=10,  
scoring='neg_mean_squared_error', cv=5, n_jobs=-1, random_state=2)
```

4. Now fit the model on the training set and obtain the best parameters and scores:

```
rand_reg.fit(X_train, y_train)  
best_model = rand_reg.best_estimator_  
best_params = rand_reg.best_params_  
print("Best params:", best_params)  
best_score = np.sqrt(-rand_reg.best_score_)  
print("Training score: {:.3f}".format(best_score))  
y_pred = best_model.predict(X_test)  
rmse_test = MSE(y_test, y_pred)**0.5  
print('Test set score: {:.3f}'.format(rmse_test))
```

The result is as follows:

```
Best params: {'learning_rate': 0.05, 'n_estimators': 300, 'subsample':  
0.65}  
Training score: 636.200  
Test set score: 625.985
```

From here, it's worth experimenting by changing parameters individually or in pairs.

Even though the best model currently has **n_estimators=300**, it's certainly possible that raising this hyperparameter will obtain better results with careful adjustment of the **learning_rate** value. **subsample** may be experimented with as well.

5. After a few rounds of experimentation, we obtained the following model:

```
gbr = GradientBoostingRegressor(max_depth=3, n_estimators=1600,  
subsample=0.75, learning_rate=0.02, random_state=2)  
gbr.fit(X_train, y_train)  
y_pred = gbr.predict(X_test)  
MSE(y_test, y_pred)**0.5
```

The result is the following:

```
596.9544588974487
```

With a larger value for **n_estimators** at 1600, a smaller **learning_rate** value at 0.02, a comparable **subsample** value of 0.75, and the same **max_depth** value of 3, we obtained the best **Root Mean Square Error (RMSE)** yet at 597.

It may be possible to do better. We encourage you to try!

Now, let's see how XGBoost differs from gradient boosting using the same hyperparameters covered thus far.

XGBoost

XGBoost is an advanced version of gradient boosting with the same general structure, meaning that it transforms weak learners into strong learners by summing the residuals of trees.

The only difference in hyperparameters from the last section is that XGBoost refers to `learning_rate` as `eta`.

Let's build an XGBoost regressor with the same hyperparameters to compare the results.

Import `XGBRegressor` from `xgboost`, and then initialize and score the model as follows:

```
from xgboost import XGBRegressor
xg_reg = XGBRegressor(max_depth=3, n_estimators=1600, eta=0.02, subsample=0.75,
random_state=2)
xg_reg.fit(X_train, y_train)
y_pred = xg_reg.predict(X_test)
MSE(y_test, y_pred)**0.5
```

The result is this:

```
584.339544309016
```

The score is better. The reason as to why the score is better will be revealed in the next chapter, [*Chapter 5, XGBoost Unveiled*](#).

Accuracy and speed are the two most important concepts when building machine learning models, and we have shown multiple times that XGBoost is very accurate. XGBoost is preferred over gradient boosting in general because it consistently delivers better results, and because it's faster, as demonstrated by the following case study.

Approaching big data – gradient boosting versus XGBoost

In the real world, datasets can be enormous, with trillions of data points. Limiting work to one computer can be disadvantageous due to the limited resources of one machine. When working with big data, the cloud is often used to take advantage of parallel computers.

Datasets are big when they push the limits of computation. So far in this book, by limiting datasets to tens of thousands of rows with a hundred or fewer columns, there should have been no significant time delays, unless you ran into errors (happens to everyone).

In this section, we examine **exoplanets** over time. The dataset has 5,087 rows and 3,189 columns that record light flux at different times of a star's life cycle. Multiplying columns and

rows together results in 1.5 million data points. Using a baseline of 100 trees, we need 150 million data points to build a model.

In this section, my 2013 MacBook Air had wait times of about 5 minutes. New computers should be faster. I have chosen the exoplanet dataset so that wait times play a significant role without tying up your computer for a very long time.

Introducing the exoplanet dataset

The exoplanet dataset is taken from Kaggle and dates from around 2017: <https://www.kaggle.com/keplersmachines/kepler-labelled-time-series-data>. The dataset contains information about the light of stars. Each row is an individual star and the columns reveal different light patterns over time. In addition to light patterns, an exoplanet column is labeled 2 if the star hosts an exoplanet; otherwise, it is labeled 1.

The dataset records the light flux from thousands of stars. **Light flux**, often referred to as **luminous flux**, is the perceived brightness of a star.

Note

The perceived brightness is different than actual brightness. For instance, an incredibly bright star very far away may have a small luminous flux (looks dim), while a moderately bright star that is very close, like the sun, may have a large luminous flux (looks bright).

When the light flux of an individual star changes periodically, it is possible that the star is being orbited by an exoplanet. The assumption is that when an exoplanet orbits in front of a star, it blocks a small fraction of the light, reducing the perceived brightness by a very slight amount.

Tip

Finding exoplanets is rare. The predictive column, on whether a star hosts an exoplanet or not, has very few positive cases, resulting in an imbalanced dataset. Imbalanced datasets require extra precautions. We will cover imbalanced datasets in [Chapter 7](#), Discovering Exoplanets with XGBoost, where we go into further detail with this dataset.

Next, let's access the exoplanet dataset and prepare it for machine learning.

Preprocessing the exoplanet dataset

The exoplanet dataset has been uploaded to our GitHub page at <https://github.com/PacktPublishing/Hands-On-Gradient-Boosting-with-XGBoost-and-Scikit-learn/tree/master/Chapter04>.

Here are the steps to load and preprocess the exoplanet dataset for machine learning:

1. Download `exoplanets.csv` in the same folder as your Jupyter Notebook. Then, open the file and take a look:

```
df = pd.read_csv('exoplanets.csv')
df.head()
```

The DataFrame will look as shown in the following figure:

	LABEL	FLUX.1	FLUX.2	FLUX.3	FLUX.4	FLUX.5	FLUX.6	FLUX.7	FLUX.8	FLUX.9
0	2	93.85	83.81	20.10	-26.98	-39.56	-124.71	-135.18	-96.27	-79.89
1	2	-38.88	-33.83	-58.54	-40.09	-79.31	-72.81	-86.55	-85.33	-83.97
2	2	532.64	535.92	513.73	496.92	456.45	466.00	464.50	486.39	436.56
3	2	326.52	347.39	302.35	298.13	317.74	312.70	322.33	311.31	312.42
4	2	-1107.21	-1112.59	-1118.95	-1095.10	-1057.55	-1034.48	-998.34	-1022.71	-989.57

5 rows × 3198 columns

Fig 4.5 – Exoplanet DataFrame

Not all columns are shown due to space limitations. The flux columns are floats, while the `Label` column is 2 for an exoplanet star and 1 for a non-exoplanet star.

2. Let's confirm that all columns are numerical with `df.info()`:

```
df.info()
```

The result is as follows:

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 5087 entries, 0 to 5086
Columns: 3198 entries, LABEL to FLUX.3197
dtypes: float64(3197), int64(1)
memory usage: 124.1 MB
```

As you can see from the output, 3197 columns are floats and 1 column is an `int`, so all columns are numerical.

3. Now, let's confirm the number of null values with the following code:

```
df.isnull().sum().sum()
```

The output is as follows:

```
0
```

The output reveals that there are no null values.

4. Since all columns are numerical with no null values, we may split the data into training and test sets. Note that the 0th column is the target column, `y`, and all other columns are the predictor columns, `x`:

```
X = df.iloc[:,1:]
y = df.iloc[:,0]
X_train, X_test, y_train, y_test = train_test_split(X, y, random_state=2)
```

It's time to build a gradient boosting classifier to predict whether stars host exoplanets.

Building gradient boosting classifiers

Gradient boosting classifiers work in the same manner as gradient boosting regressors. The difference is primarily in the scoring.

Let's start by importing **GradientBoostingClassifier** and **XGBClassifier** in addition to **accuracy_score** so that we may compare both models:

```
from sklearn.ensemble import GradientBoostingClassifier
from xgboost import XGBClassifier
from sklearn.metrics import accuracy_score
```

Next, we need a way to compare models using a timer.

Timing models

Python comes with a **time** library that can be used to mark time. The general idea is to mark the time before and after a computation. The difference between these times tells us how long the computation took.

The **time** library is imported as follows:

```
import time
```

Within the **time** library, the **.time()** method marks time in seconds.

As an example, see how long it takes to run **df.info()** by assigning start and end times before and after the computation using **time.time()**:

```
start = time.time()
df.info()
end = time.time()
elapsed = end - start
print('\nRun Time: ' + str(elapsed) + ' seconds.')
```

The output is as follows:

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 5087 entries, 0 to 5086
Columns: 3198 entries, LABEL to FLUX.3197
dtypes: float64(3197), int64(1)
memory usage: 124.1 MB
```

The runtime is as follows:

```
Run Time: 0.0525362491607666 seconds.
```

Your results will differ from ours, but hopefully it's in the same ballpark.

Let's now compare **GradientBoostingClassifier** and **XGBoostClassifier** with the exoplanet dataset for its speed using the preceding code to mark time.

Tip

Jupyter Notebooks come with magic functions, denoted by the `%` sign before a command. `%timeit` is one such magic function. Instead of computing how long it takes to run the code once, `%timeit` computes how long it takes to run code over multiple runs.

See ipython.readthedocs.io/en/stable/interactive/magics.html for more information on magic functions.

Comparing speed

It's time to race `GradientBoostingClassifier` and `XGBoostClassifier` with the exoplanet dataset. We have set `max_depth=2` and `n_estimators=100` to limit the size of the model. Let's start with `GradientBoostingClassifier`:

1. First, we will mark the start time. After building and scoring the model, we will mark the end time. The following code may take around 5 minutes to run depending on the speed of your computer:

```
start = time.time()
gbr = GradientBoostingClassifier(n_estimators=100, max_depth=2,
random_state=2)
gbr.fit(X_train, y_train)
y_pred = gbr.predict(X_test)
score = accuracy_score(y_pred, y_test)
print('Score: ' + str(score))
end = time.time()
elapsed = end - start
print('\nRun Time: ' + str(elapsed) + ' seconds')
```

The result is this:

```
Score: 0.9874213836477987
Run Time: 317.6318619251251 seconds
```

`GradientBoostingRegressor` took over 5 minutes to run on my 2013 MacBook Air. Not bad for 150 million data points on an older computer.

Note

While a score of 98.7% percent is usually outstanding for accuracy, this is not the case with imbalanced datasets, as you will see in [Chapter 7](#), *Discovering Exoplanets with XGBoost*.

2. Next, we will build an `XGBClassifier` model with the same hyperparameters and mark the time in the same manner:

```
start = time.time()
xg_reg = XGBClassifier(n_estimators=100, max_depth=2, random_state=2)
xg_reg.fit(X_train, y_train)
y_pred = xg_reg.predict(X_test)
score = accuracy_score(y_pred, y_test)
print('Score: ' + str(score))
end = time.time()
elapsed = end - start
```

```
print('Run Time: ' + str(elapsed) + ' seconds')
```

The result is as follows:

```
Score: 0.9913522012578616
```

```
Run Time: 118.90568995475769 seconds
```

On my 2013 MacBook Air, XGBoost took under 2 minutes, making it more than twice as fast. It's also more accurate by half a percentage point.

When it comes to big data, an algorithm twice as fast can save weeks or months of computational time and resources. This advantage is huge in the world of big data.

In the world of boosting, XGBoost is the model of choice due to its unparalleled speed and impressive accuracy.

As for the exoplanet dataset, it will be revisited in [Chapter 7, Discovering Exoplanets with XGBoost](#), in an important case study that reveals the challenges of working with imbalanced datasets along with a variety of potential solutions to those challenges.

Note

I recently purchased a 2020 MacBook Pro and updated all software. The difference in time using the same code is staggering:

Gradient Boosting Run Time: 197.38 seconds

XGBoost Run Time: 8.66 seconds

More than a 10-fold difference!

Summary

In this chapter, you learned the difference between bagging and boosting. You learned how gradient boosting works by building a gradient boosting regressor from scratch. You implemented a variety of gradient boosting hyperparameters, including `learning_rate`, `n_estimators`, `max_depth`, and `subsample`, which results in stochastic gradient boosting. Finally, you used big data to predict whether stars have exoplanets by comparing the times of `GradientBoostingClassifier` and `XGBoostClassifier`, with `XGBoostClassifier` emerging as twice to over ten times as fast and more accurate.

The advantage of learning these skills is that you now understand when to apply XGBoost rather than similar machine learning algorithms such as gradient boosting. You can now build stronger XGBoost and gradient boosting models by properly taking advantage of core

hyperparameters, including `n_estimators` and `learning_rate`. Furthermore, you have developed the capacity to time all computations instead of relying on intuition.

Congratulations! You have completed all of the preliminary XGBoost chapters. Until now, the purpose has been to introduce you to machine learning and data analytics within the larger XGBoost narrative. The aim has been to show how the need for XGBoost emerged from ensemble methods, boosting, gradient boosting, and big data.

The next chapter starts a new leg on our journey with an advanced introduction to XGBoost, where you will learn the mathematical details behind the XGBoost algorithm in addition to hardware modifications that XGBoost makes to improve speed. You'll also be building XGBoost models using the original Python API in a historically relevant case study on the discovery of the Higgs boson. The chapters that follow highlight exciting details, advantages, nuances, and tricks and tips to build swift, efficient, powerful, and industry-ready XGBoost models that you can use for years to come.